

Email Generation Assistant

AI Engineer Candidate Assessment (Final Report)

Advanced Prompt Engineering, Custom Evaluation Metrics, and Comparative Model Analysis

Prepared by: **Fahim Shahriar**

Date: 20 June 2026

Repository: [GitHub](#)

Demo video: [Play](#)

1. Prompt Template

The assistant generates emails using a hybrid strategy combining three advanced prompting techniques: Role-Playing, Few-Shot Examples, and Chain-of-Thought (CoT) reasoning. The full implementation lives in `backend/graph/prompt_engineering.py`. Two distinct prompt families are used in the system: the generation prompt (sent to the email-writing model) and the judge prompts (sent to the LLM evaluator during scoring). Both are included below.

1.1 Generation Prompt — System Message (Role-Playing)

Casts the model as a senior communication director with an explicit professional code: every supplied fact must appear, tone is treated as a first-class constraint rather than decoration, and structure follows a fixed convention (greeting, context, body, ask/close, sign-off). The system message also instructs the model to reason inside `<thinking>` tags before producing the final email inside `<email>` tags.

“You are a senior communication director with 20 years of experience crafting high-stakes professional emails for Fortune 500 executives, diplomatic missions, and fast-moving startups. Your emails are precise, human, and perfectly calibrated — readers consistently say they feel understood, not processed.

You operate by a personal code:

- *Every fact the sender provides must appear in the email — no omissions, no invented details.*
- *Tone is not decoration; it is the message. A formal email uses a different lexicon than a casual one, not just a different sign-off.*
- *Structure matters: greeting → context → body → clear ask or close → sign-off. Short, scannable paragraphs.*
- *You never pad. If the email can be shorter and still achieve its goal, *
it is shorter.

When you receive a request you will ALWAYS think through it step-by-step inside <thinking> tags before writing the email. Your thinking must cover:

- 1. What is the core goal of this email?*
- 2. List every fact that must appear and where it fits best.*
- 3. What vocabulary, sentence length, and register does this tone demand?*
- 4. What is the ideal structure for this specific email?*

After your thinking, produce the final email inside <email> tags.”

1.2 Generation Prompt — User Message (Few-Shot + Chain-of-Thought)

The user message embeds two fully worked few-shot examples (one formal, one casual), each demonstrating the complete expected format: input, four-point step-by-step reasoning, and final structured email. The actual scenario inputs (intent, key facts, tone) are appended after the examples, along with an explicit restatement of the CoT and fact-inclusion requirements.

“EXAMPLE 1: FORMAL TONE

INPUT

Intent: Follow up on a pending contract signature from a client

Key Facts:

- *Contract sent on June 3rd*

- *Deadline for project kickoff is June 20th*
- *Client is Meridian Logistics*
- *Contact is their procurement manager, David Holt*

Tone: formal

RESPONSE

<thinking>

1. Goal: nudge David Holt to sign without sounding impatient or aggressive.
 2. Facts to include: contract sent June 3 → still unsigned; kickoff deadline June 20 → creates natural urgency; recipient is David Holt, procurement manager, Meridian Logistics.
 3. Formal tone demands: full salutation ("Dear Mr. Holt"), no contractions, Latinate vocabulary where natural ("ensure", "confirm", "proceed"), measured sentence length, professional sign-off ("Kind regards" or "Sincerely").
 4. Structure: opener acknowledging the sent contract → polite urgency framed around the June 20 deadline → clear ask (sign / confirm receipt) → offer to assist → sign-off.
 </thinking>
 <email>
 Subject: Follow-Up: Contract Signature Required — Kickoff Deadline June 20

Dear Mr. Holt,

I hope this message finds you well. I am writing to follow up on the contract we sent to Meridian Logistics on June 3rd, which we have not yet received signed.

With our project kickoff scheduled for June 20th, we are approaching the window within which all documentation must be finalised to ensure a smooth start. We want to make certain there are no obstacles on your end that we can help address.

Could you please confirm receipt of the contract and advise on an expected signature date? If you have questions or require any amendments, I am available at your convenience.

Thank you for your time and continued partnership.

Kind regards,
 [Your Name]
 [Your Title]
 [Your Contact Information]
 </email>

EXAMPLE 2: CASUAL TONE

INPUT

Intent: Invite a colleague to a team lunch celebrating a product launch

Key Facts: • Launch was for "Spark 2.0"

- Lunch is Friday, June 21st at 12:30 PM
- Venue is The Noodle House on 5th Ave
- RSVP by Wednesday EOD

Tone: casual

RESPONSE

<thinking>

1. Goal: get Sarah (or whoever) excited and to RSVP quickly — low friction, warm vibe.
2. Facts: Spark 2.0 launch (celebrate this — lead with it), Friday June 21, 12:30 PM, The Noodle House on 5th Ave, RSVP by Wednesday EOD.
3. Casual tone demands: contractions ("we're", "it's"), conversational opener, first names, emoji optional but one is fine, short punchy sentences, informal sign-off ("Cheers", "See you there").
4. Structure: hook on the win → invite details inline → RSVP ask → cheerful close.

</thinking>

<email>

Subject: Lunch on us — Spark 2.0 launch celebration

Hey [Name],

We shipped Spark 2.0 and it's time to celebrate properly! Join us for a team lunch this Friday, June 21st at 12:30 PM at The Noodle House on 5th Ave.

Good food, the whole crew, and zero Slack notifications for an hour — sounds pretty good, right?

Just RSVP by Wednesday EOD so we can sort out the reservation. Hope to see you there!

*Cheers,
[Your Name]
</email>*

NOW YOUR TASK

INPUT

*Intent: {intent}
Key Facts: {key_facts_formatted}
Tone: {tone}*

Remember:

- *Think step-by-step inside <thinking> tags first (cover all 4 points from your instructions).*
- *Then produce the final, polished email inside <email> tags.*
- *The email must include EVERY key fact — omitting any fact is a critical error.*
- *Match the tone precisely: vocabulary, register, sentence rhythm, sign-off must all reflect "{tone}".*
- *Do not add invented facts, names, or details not present in the input."*

1.3 LLM-as-Judge Prompt: Tone Adherence (Metric 2)

A separate, fixed evaluator prompt used only during evaluation, never during generation. Gives the judge model the target tone and the generated email, and requests a rubric-anchored 1–5 score plus a one-line justification, returned as strict JSON.

“You are an expert writing evaluator specializing in tone analysis. You will be given a target tone and a generated email. Rate how well the email's actual tone matches the target tone using this rubric:

5 = Tone is unmistakable and consistent throughout; word choice, sentence structure, and register are all clearly aligned with the target tone.

4 = Tone is mostly aligned with only minor lapses.

3 = Tone is mixed/inconsistent, or generic/neutral rather than clearly hitting the target.

2 = Tone is mostly wrong, with only incidental alignment.

1 = Tone is opposite or unrelated to what was requested.

Respond with ONLY a JSON object (no markdown fences, no preamble), shaped exactly like:

{"score": <integer 1-5>, "justification": "one to two short sentences"}

1.4 Why This Combination

- **Role-Playing + Few-Shot:** Improves Tone Adherence and Structural Completeness by giving the model a consistent persona and two complete worked examples of the expected register and layout.
- **Chain-of-Thought:** Improves Fact Recall by forcing an explicit fact-enumeration step ("list every fact and where it fits") before any prose is generated, rather than relying on the model to track facts implicitly while drafting.

- **Structured tags** (<thinking> / <email>): Make the reasoning trace separable from the deliverable output via a simple parser (extract_cot_and_email), so the CoT is available for debugging and auditing without ever leaking into the email shown to the end user.

2. Custom Evaluation Metrics (Definitions and Logic)

Three custom metrics were designed specifically for email generation quality, each targeting an independent dimension of output quality: content (facts), style (tone), and form (structure). The three are intentionally orthogonal by construction — an email can include every requested fact and still fail on tone, or read fluently while missing a sign-off. No single metric can compensate for failure on another, which is the point: a composite score built from independent axes is far more diagnostic than one blended judgment call. The ten test scenarios were deliberately written to be more demanding than a typical email-assistant benchmark to make the task more challenging for LLMs.

2.1 Metric 1: Fact Recall (Embedding Cosine Similarity)

What it measures: of the user-supplied key facts, how well the generated email's content semantically covers each one.

Logic: each key fact and each sentence-level chunk of the generated email is embedded with OpenAI's text-embedding-3-small. For every fact, the maximum cosine similarity across all email chunks is taken (max-pooling — a fact present anywhere in the email should score well regardless of where it appears). The overall score for a scenario is the mean of all per-fact similarity scores, which is a continuous quantity, not simply the binary found/not-found fraction.

Why: Embedding similarity also has a methodological advantage beyond cost (It is graded by construction). Two facts that are both "present" are not necessarily expressed with equal clarity — one might be stated directly, another buried in a subordinate clause. Mean cosine similarity captures that difference; a binary judge verdict does not.

2.2 Metric 2: Tone Adherence (LLM-as-Judge)

What it measures: whether the email's actual register, word choice, and sentence structure match the requested tone — independent of whether the email is well-written in some generic sense.

Logic: GPT-4o-mini is given the target tone and the generated email, and scores tone match on a rubric-anchored 1–5 Likert scale (5 = unmistakable and consistent tone alignment throughout; 1 = opposite or unrelated to the requested tone). The raw score is normalized to [0, 1] via $(\text{score} - 1) / 4$ for direct comparability and averaging with the other two metrics.

Why LLM-as-judge here, specifically: tone is a holistic, partly subjective stylistic judgment — formality of vocabulary, sentence length and rhythm, contraction use, directness, warmth — that does not reduce well to a regex or keyword check the way structural completeness does. A rubric-anchored LLM judge with a fixed scoring scale is the practical, standard way to operationalize this kind of qualitative assessment at scale while keeping it reproducible.

2.3 Metric 3: Structural Completeness (Rule-Based)

What it measures: whether the generated output has the shape of a real, sendable email — independent of its factual or tonal quality.

Logic: five binary checks, each worth 0.2 of the final score:

- Has a "Subject:" line near the top of the output
- Has a recognizable greeting ("Dear", "Hi", "Hello", etc.)
- Has a non-trivial body — at least 25 words of content
- Has a recognizable sign-off ("Regards", "Sincerely", "Best", "Cheers", etc.)
- Contains no leaked chain-of-thought tags, AI self-reference, or unfilled template placeholders

Why rule-based and not LLM-based: this is a fully deterministic check — every input is a known string processed by regular expressions, with no external or ambiguous judgment required. Routing this through an LLM call would add latency and cost with no accuracy benefit over a small set of well-tested pattern checks. It is also the cheapest metric in the entire pipeline (zero API calls) and the only one that runs instantly, making it useful as a fast pre-check during development before spending API budget on the other two.

2.4 Composite Score

The unweighted mean of the three metrics above, computed per scenario and averaged across all 10 scenarios per model, reported in the range [0, 1].

3. Raw Evaluation Data

Both models were evaluated against the identical 10 scenarios using the identical evaluation pipeline. The LLM judge (Tone Adherence) and embedding model (Fact Recall) were held fixed at GPT-4o-mini and text-embedding-3-small for both runs, ensuring the comparison isolates generation quality rather than judge variance. Full machine-readable output is available in evaluation/results/ (model_a_results.csv, model_b_results.csv, and the corresponding _detail.json files with per-fact similarity scores and judge justifications).

3.1 Aggregate Metric Performance

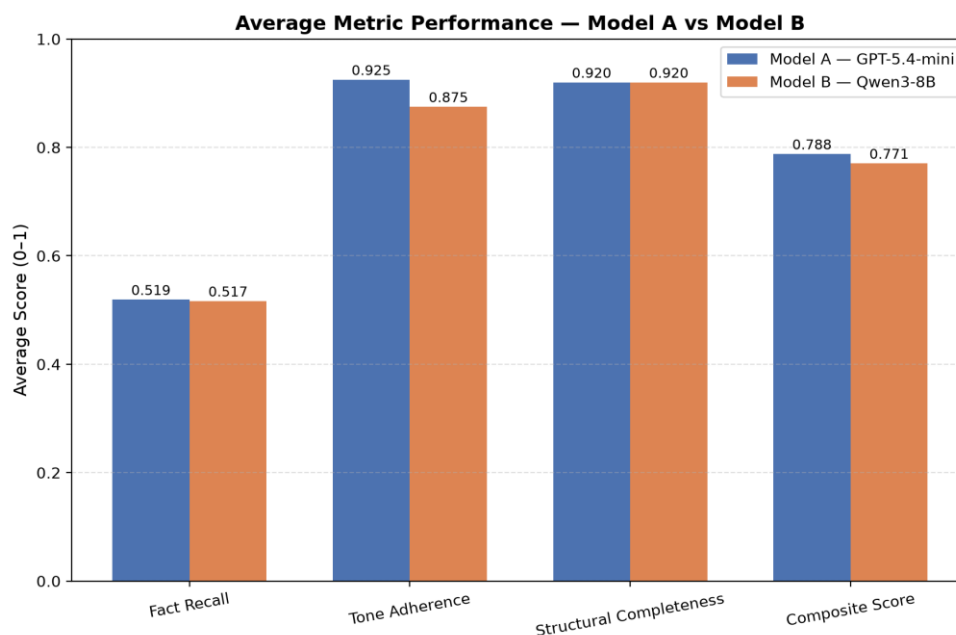


Figure 1 — Average score per metric across all 10 scenarios, Model A (GPT-5.4-mini) vs Model B (Qwen3-8B).

3.2 Composite Score by Scenario

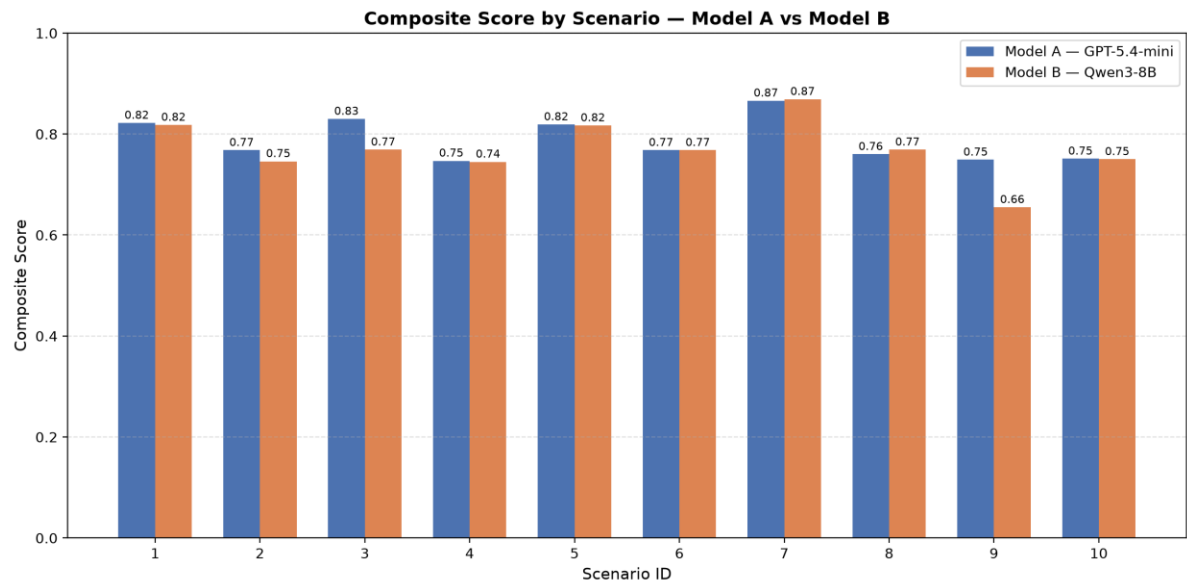


Figure 2 — Per-scenario composite score (mean of all three metrics), Model A vs Model B.

3.3 Per-Scenario Metric Breakdown

The two charts above report composite scores. The table below decomposes each scenario into its three individual metric scores per model, to make visible where the two models diverge or agree at the metric level rather than only at the blended composite level.

ID	Tone	Fact Recall	Tone Adherence	Structural	Fact Recall	Tone Adherence	Structural
Scenario		Model A — GPT-5.4-mini			Model B — Qwen3-8B (Local)		
1	Formal	0.467	1.00	1.0	0.453	1.00	1.0
2	Empathetic	0.553	0.75	1.0	0.487	0.75	1.0
3	Persuasive	0.489	1.00	1.0	0.556	0.75	1.0
4	Urgent	0.642	1.00	0.6	0.635	1.00	0.6
5	Friendly	0.457	1.00	1.0	0.450	1.00	1.0
6	Empathetic	0.555	0.75	1.0	0.553	0.75	1.0
7	Formal	0.598	1.00	1.0	0.607	1.00	1.0
8	Assertive	0.531	0.75	1.0	0.558	0.75	1.0
9	Casual	0.447	1.00	0.8	0.416	0.75	0.8
10	Persuasive	0.454	1.00	0.8	0.451	1.00	0.8

Table 1 — Per-scenario scores for all three custom metrics. Fact Recall in [0,1] (mean cosine similarity); Tone Adherence normalized to [0,1]; Structural Completeness in [0,1] (fraction of five checks passed).

3.4 Summary Statistics

Metric	Model A (GPT-5.4-mini)	Model B (Qwen3-8B)	Difference (A – B)
Fact Recall (avg)	0.519	0.517	+0.002
Tone Adherence (avg, 0–1)	0.925	0.875	+0.050
Structural Completeness (avg)	0.920	0.920	0.000
Composite Score (avg)	0.788	0.771	+0.017
Success rate	10 / 10	10 / 10	—

Table 2 — Aggregate summary across all 10 scenarios.

4. Comparative Analysis

4.1 Which model/strategy performed better

On the unweighted composite score, Model A (GPT-5.4-mini) edges out Model B (Qwen3-8B, run locally) by 0.788 to 0.771 — a 0.017 gap. Decomposed by metric, this gap is almost entirely attributable to Tone Adherence (0.925 vs 0.875, a 0.050 gap); Fact Recall is effectively tied (0.519 vs 0.517) and Structural Completeness is identical (0.920 vs 0.920). Both models succeeded on all 10 scenarios with no generation failures. In short: GPT-5.4-mini is the stronger model on this benchmark, but the margin is narrow and concentrated in a single dimension out of three.

4.2 Biggest failure mode of the lower-performing model

Qwen3-8B's weakest area is tone consistency, not content coverage or structural conformity. The per-scenario breakdown (Table 1) shows Qwen3-8B dropping to the lower tone-adherence band (0.75 rather than 1.00) on scenario 9 (casual tone) where GPT-5.4-mini held the top band, and matching GPT-5.4-mini's lower band on most of the other mid-register scenarios (empathetic, assertive). This pattern — fact inclusion and structural conventions holding up reliably while tonal register is the dimension most likely to drift — is consistent with a smaller model's instruction-following being most fragile on subjective stylistic constraints (vocabulary choice, sentence rhythm, warmth) rather than on the more mechanically checkable requirements (a fact is either present or absent; a sign-off either exists or does not). Fact Recall and Structural Completeness were demonstrated to hold up reliably even on the deliberately demanding scenario set described in Section 2, which is itself informative: the gap between these two models is not a capability gap so much as a tone-calibration gap.

4.3 Production recommendation

Recommendation: Qwen3-8B (Apache License 2.0), self-hosted, is recommended for production use in this email-generation assistant.

This recommendation is driven primarily by cost and deployment economics relative to the size of the performance gap actually observed:

- **The performance gap is narrow and concentrated.** A 0.017 composite-score gap, entirely attributable to a 0.050 gap on one of three metrics, while two of the three custom metrics (Fact Recall, Structural Completeness) show no meaningful difference between the models. This is not a quality gap that justifies a large recurring cost difference.
- **API cost asymmetry is substantial.** GPT-5.4-mini is priced at \$0.75 per million input tokens and \$4.50 per million output tokens. Qwen3-8B run locally has zero marginal per-request API cost beyond the fixed cost

of the GPU it runs on — for any deployment with meaningful sustained request volume, this difference compounds directly into operating margin.

- **Hardware footprint is modest.** Qwen3-8B at 4-bit quantization was run for this evaluation on a single consumer-grade RTX 3070 Ti with 8GB of VRAM, using roughly 5–6GB of that budget. This is a low bar for self-hosting — it does not require enterprise-grade GPU infrastructure, multi-GPU sharding, or specialized inference hardware.
- **Open licensing removes deployment constraints.** Apache License 2.0 permits unrestricted commercial self-hosting, fine-tuning, and redistribution, with no per-token billing, no dependency on a third-party API's uptime or rate limits, and no data leaving the deployment environment — a meaningful advantage for any use case with data-residency or privacy requirements around the email content being generated (which may contain sensitive business or personal information).

The counter-argument for GPT-5.4-mini is real but narrow: if tone fidelity is the single most important quality axis for a given product (for example, a brand-voice-sensitive customer-facing tool where a 5-point tone-adherence gap is unacceptable regardless of cost), the API-based model remains the safer default. But for the general-purpose internal/professional email-generation use case this assessment targets, the cost-to-quality trade strongly favors the self-hosted open-weight model — the residual quality gap is small enough to be addressed through targeted prompt refinement on lower-scoring tones (notably casual and empathetic) rather than by accepting a recurring per-token API cost for a marginal, single-metric improvement.

Appendix: Evaluation Configuration

- **Generator, Model A:** gpt-5.4-mini-2026-03-17 (OpenAI API)
- **Generator, Model B:** qwen3:8b, 4-bit quantization, served locally via Ollama (OpenAI-compatible endpoint), tested on an RTX 3070 Ti / 8GB VRAM
- **LLM-as-judge (Tone Adherence):** gpt-4o-mini, held fixed across both model evaluation runs
- **Embedding model (Fact Recall):** text-embedding-3-small, held fixed across both model evaluation runs
- **Test set:** 10 fixed scenarios, evaluation/scenarios.json, each with an intent, 3–5 key facts, a target tone, and a human-written reference email
- **Orchestration:** LangGraph pipeline (validate_input → build_prompt → generate_email), identical code path used for both live API serving and evaluation
- **Full raw output:** evaluation/results/model_a_results.csv, model_b_results.csv, model_a_detail.json, model_b_detail.json.